3 Numerical methods and comparison protocol

Section 2 showed that the supported matrix A is sparse, symmetric and positive definite, with sensitivity measured by $\kappa_2(A)$. We now use these properties to choose solvers and fix one protocol for comparing them. Let $N = |F|$ be the number of unknowns, x_k the k-th iterate, $e_k = x - x_k$ its error and $r_k = b - Ax_k$ its residual.

3.1 Direct methods

A direct method factorizes A into triangular factors: $PA = LU$ with a row permutation P, or, for symmetric positive definite A, Cholesky $A = LL^T$ without pivoting at half the cost [10]. Dense factorization costs $O(N^3)$ operations but each later solve only $O(N^2)$; forming the inverse costs more and is less accurate. In sparse A, elimination creates *fill-in*, nonzeros of the factors where A has zeros. SciPy's `splu` limits it with a column ordering (COLAMD) before LU with partial pivoting [8]; factorization dominates the cost, and each solve is two triangular substitutions.

3.2 Stationary iterations

Iterative methods need only products with A. A stationary method corrects the iterate with the residual, using an easily inverted approximation M of A:

$$x_{k+1} = x_k + M^{-1}(b - Ax_k), \qquad e_{k+1} = (I - M^{-1}A)\, e_k. \tag{10}$$

It converges for every x_0 if and only if the spectral radius $\rho(I - M^{-1}A)$ is below one [4]. *Jacobi* uses $M = D_A$, the diagonal of A. Its iteration matrix is similar to $I - S$, where $S = D_A^{-1/2} A D_A^{-1/2}$, so

$$\rho_J = \max_i |1 - \lambda_i(S)|. \tag{11}$$

Positive definiteness only gives $\lambda_i(S) > 0$, so Jacobi diverges when $\lambda_{\max}(S) > 2$. *Damped Jacobi*, $M = D_A/\omega$, converges if and only if $0 < \omega < 2/\lambda_{\max}(S)$; we use $\omega = 0.5$ and check this bound for each system. *Gauss–Seidel* takes M as the lower triangle of A, including the diagonal, so each sweep is one forward substitution. It converges for every symmetric positive definite A [4].

3.3 Krylov methods

For symmetric positive definite A, the solution of $Ax = b$ minimizes $\phi(y) = \frac{1}{2} y^T A y - b^T y$, the elastic energy of Eq. (8) minus the work of the loads, and $r = -\nabla\phi$. *Steepest descent* (SD) moves along r_k to the minimum of ϕ on that line. Each step multiplies its error in the energy norm $\|e\|_A = (e^T A e)^{1/2}$ by at most $(\kappa-1)/(\kappa+1)$, where $\kappa = \kappa_2(A)$ [9]. The *conjugate gradient* (CG) method instead minimizes $\|e_k\|_A$ over all of $x_0 + \mathcal{K}_k$, where $\mathcal{K}_k = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}$ is the Krylov space. It needs one product with A per iteration and satisfies [9]

$$\|e_k\|_A \le 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A, \tag{12}$$

so its iteration count grows with $\sqrt{\kappa}$ rather than κ. This matters because, as for the model Poisson problem, κ grows like h^{-2} on quasi-uniform meshes of size h [1].

Diagonally preconditioned CG (PCG) uses D_A as preconditioner and is equivalent to CG on S, so $\kappa_2(S)$ replaces κ in Eq. (12). If the diagonal is nearly constant, S is nearly a multiple of A and nothing is gained. Two-dimensional element stiffness does not scale with element size, so the diagonal varies only with the number and shape of elements at a node and with material or thickness. *MINRES* minimizes $\|r_k\|_2$ over $x_0 + \mathcal{K}_k$ for symmetric A [2]; *GMRES* does so for general A [3], but must store and orthogonalize against all k basis vectors, costing $O(kN)$ memory and work at step k. For symmetric A both give the same iterates in exact arithmetic, so full GMRES (restart length N) measures the cost of ignoring symmetry. CG, PCG and MINRES exploit this structure; the other methods are baselines predicted by ρ_J and κ.

3.4 Comparison protocol

Every method receives the same A and b and starts from $x_0 = 0$. A returned vector $\hat{x}$ is accepted only if its recomputed relative residual satisfies

$$\eta(\hat{x}) = \frac{\|b - A\hat{x}\|_2}{\|b\|_2} < 10^{-8}. \tag{13}$$

Native solver flags are recorded but never replace this test, since SciPy's Krylov solvers stop on internal residual estimates [6, 7]; MINRES gets native tolerance zero and stops once Eq. (13) holds. An iteration is a sweep for the stationary methods, a search direction for SD, CG, PCG and MINRES, and an inner Arnoldi step for GMRES, which never restarts. The cap is 20,000 iterations, and ordinary Jacobi also stops if $\eta > 10^{10}$ or becomes non-finite. Hitting the cap or this divergence guard counts as failure; the status, iteration count and final η are still reported.

Accuracy is the relative error $\|\hat{x} - x_{\text{ref}}\|_2 / \|x_{\text{ref}}\|_2$ against the sparse LU solution x_{ref} of the same system. As $\hat{x}$ solves the system with load perturbed by its residual, the standard bound [10]

$$\frac{\|\delta x\|_2}{\|x\|_2} \le \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} \tag{14}$$

limits this error to $\kappa_2(A)\,\eta$. This *algebraic* error measures how well one mesh's equations are solved. *Discretization* error, between finite element and continuum displacements, does not shrink with the tolerance; we assess it by comparing meshes at matched physical points under the same physical load.

Times are the best of three wall-clock runs (`perf_counter`). Sparse LU factorization and solve are timed separately. Iterative times include method-specific setup and stopping checks, but exclude assembly, reduction, eigenvalue diagnostics, the reference solve and residual histories, which come from a separate run. They compare these SciPy implementations on one machine, not intrinsic algorithm costs. The small worked system also compares dense $PA = LU$, the explicit inverse and Cholesky. The eigenvalues in Eq. (9) come from dense `eigvalsh` for small systems, otherwise from `eigsh` with tolerance 10^{-10}: `which='LA'` for $\lambda_{\max}$ and shift-invert about zero for $\lambda_{\min}$ [5]. Applied to S, they give ρ_J and the damped-Jacobi bound.

Sections 4 and 6 also check that applied loads and the reactions of Eq. (7) have zero resultant force and moment, up to rounding, and that d_C equals the prescribed values. An affine patch test restrains the left edge only horizontally and the corner at the origin vertically, with uniform traction p on the opposite edge; the plane-stress solution $u_x = px/E$, $u_y = -\nu py/E$ is linear, so the computed nodal values must match it to rounding on any mesh. This verifies assembly, loading and reduction, not discretization error. Finally, the displacement change caused by a seeded random load perturbation is compared with the bound in Eq. (14).

References

[1] Howard C. Elman, David J. Silvester, and Andrew J. Wathen, *Finite elements and fast iterative solvers: With applications in incompressible fluid dynamics*, second ed., Oxford University Press, Oxford, 2014.

[2] C. C. Paige and M. A. Saunders, *Solution of sparse indefinite systems of linear equations*, SIAM Journal on Numerical Analysis **12** (1975), no. 4, 617–629.

[3] Youcef Saad and Martin H. Schultz, *GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems*, SIAM Journal on Scientific and Statistical Computing **7** (1986), no. 3, 856–869.

[4] Yousef Saad, *Iterative methods for sparse linear systems*, second ed., SIAM, Philadelphia, 2003.

[5] SciPy community, *scipy.sparse.linalg.eigsh*, SciPy Reference Guide, https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.eigsh.html. Accessed 10 November 2026.

[6] ———, *scipy.sparse.linalg.gmres*, SciPy Reference Guide, https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.gmres.html. Accessed 10 November 2026.

[7] ———, *scipy.sparse.linalg.minres*, SciPy Reference Guide, https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.minres.html. Accessed 10 November 2026.

[8] ———, *scipy.sparse.linalg.splu*, SciPy Reference Guide, https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.splu.html. Accessed 10 November 2026.

[9] Jonathan Richard Shewchuk, *An introduction to the conjugate gradient method without the agonizing pain*, Technical Report CMU-CS-94-125, School of Computer Science, Carnegie Mellon University, 1994, Edition $1\frac{1}{4}$.

[10] Lloyd N. Trefethen and David Bau, III, *Numerical linear algebra*, SIAM, Philadelphia, 1997.